

exactly three literals. Give a randomized 2-approximation algorithm for the MAX-CNF satisfiability problem.

35.4-3

In the MAX-CUT problem, the input is an unweighted undirected graph $G = (V, E)$. We define a cut $(S, V - S)$ as in Chapter 21 and the *weight* of a cut as the number of edges crossing the cut. The goal is to find a cut of maximum weight. Suppose that each vertex v is randomly and independently placed into S with probability $1/2$ and into $V - S$ with probability $1/2$. Show that this algorithm is a randomized 2-approximation algorithm.

35.4-4

Show that the constraints in line (35.17) are redundant in the sense that removing them from the linear-programming relaxation in lines (35.15)–(35.18) yields a linear program for which any optimal solution x must satisfy $x(v) \leq 1$ for each $v \in V$.

35.5 The subset-sum problem

Recall from Section 34.5.5 that an instance of the subset-sum problem is given by a pair (S, t) , where S is a set $\{x_1, x_2, \dots, x_n\}$ of positive integers and t is a positive integer. This decision problem asks whether there exists a subset of S that adds up exactly to the target value t . As we saw in Section 34.5.5, this problem is NP-complete.

The optimization problem associated with this decision problem arises in practical applications. The optimization problem seeks a subset of $\{x_1, x_2, \dots, x_n\}$ whose sum is as large as possible but not larger than t . For example, consider a truck that can carry no more than t pounds, which is to be loaded with up to n different boxes, the i th of which weighs x_i pounds. How heavy a load can the truck take without exceeding the t -pound weight limit?

We start this section with an exponential-time algorithm to compute the optimal value for this optimization problem. Then we show how to modify the algorithm so that it becomes a fully polynomial-time

approximation scheme. (Recall that a fully polynomial-time approximation scheme has a running time that is polynomial in $1/\epsilon$ as well as in the size of the input.)

An exponential-time exact algorithm

Suppose that you compute, for each subset S' of S , the sum of the elements in S' , and then you select, among the subsets whose sum does not exceed t , the one whose sum is closest to t . This algorithm returns the optimal solution, but it might take exponential time. To implement this algorithm, you can use an iterative procedure that, in iteration i , computes the sums of all subsets of $\{x_1, x_2, \dots, x_i\}$, using as a starting point the sums of all subsets of $\{x_1, x_2, \dots, x_{i-1}\}$. In doing so, you would realize that once a particular subset S' has a sum exceeding t , there is no reason to maintain it, since no superset of S' can be an optimal solution. Let's see how to implement this strategy.

The procedure EXACT-SUBSET-SUM takes an input set $S = \{x_1, x_2, \dots, x_n\}$, the size $n = |S|$, and a target value t . This procedure iteratively computes L_i , the list of sums of all subsets of $\{x_1, \dots, x_i\}$ that do not exceed t , and then it returns the maximum value in L_n .

If L is a list of positive integers and x is another positive integer, then let $L + x$ denote the list of integers derived from L by increasing each element of L by x . For example, if $L = \langle 1, 2, 3, 5, 9 \rangle$, then $L + 2 = \langle 3, 4, 5, 7, 11 \rangle$. This notation extends to sets, so that

$$S + x = \{s + x : s \in S\}.$$

EXACT-SUBSET-SUM (S, n, t)

```
1  $L_0 = \langle 0 \rangle$ 
2 for  $i = 1$  to  $n$ 
3    $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
4   remove from  $L_i$  every element that is greater
      than  $t$ 
5 return the largest element in  $L_n$ 
```

EXACT-SUBSET-SUM invokes an auxiliary procedure MERGE-LISTS (L, L'), which returns the sorted list that is the merge of its two sorted input lists L and L' , with duplicate values removed. Like the MERGE procedure we used in merge sort on page 36, MERGE-LISTS runs in $O(|L| + |L'|)$ time. We omit the pseudocode for MERGE-LISTS.

To see how EXACT-SUBSET-SUM works, let P_i denote the set of values obtained by selecting each (possibly empty) subset of $\{x_1, x_2, \dots, x_i\}$ and summing its members. For example, if $S = \{1, 4, 5\}$, then

$$P_1 = \{0, 1\},$$

$$P_2 = \{0, 1, 4, 5\},$$

$$P_3 = \{0, 1, 4, 5, 6, 9, 10\}.$$

Given the identity

$$P_i = P_{i-1} \cup (P_{i-1} + x_i), \quad (35.21)$$

you can prove by induction on i (see Exercise 35.5-1) that the list L_i is a sorted list containing every element of P_i whose value is not more than t . Since the length of L_i can be as much as 2^i , EXACT-SUBSET-SUM is an exponential-time algorithm in general, although it is a polynomial-time algorithm in the special cases in which t is polynomial in $|S|$ or all the numbers in S are bounded by a polynomial in $|S|$.

A fully polynomial-time approximation scheme

The key to devising a fully polynomial-time approximation scheme for the subset-sum problem is to “trim” each list L_i after it is created. Here’s the idea behind trimming: if two values in L are close to each other, then since the goal is just an approximate solution, there is no need to maintain both of them explicitly. More precisely, use a trimming parameter δ such that $0 < \delta < 1$. When *trimming* a list L by δ , remove as many elements from L as possible, in such a way that if L' is the result of trimming L , then for every element y that was removed from L , some

element z still in L' approximates y . For z to approximate y , it must be no greater than y and also within a factor of $1 + \delta$ of y , so that

$$\frac{y}{1 + \delta} \leq z \leq y. \quad (35.22)$$

You can think of such a z as “representing” y in the new list L' . Each removed element y is represented by a remaining element z satisfying inequality (35.22). For example, suppose that $\delta = 0.1$ and

$L = \langle 10, 11, 12, 15, 20, 21, 22, 23, 24, 29 \rangle$.

Then trimming L results in

$L' = \langle 10, 12, 15, 20, 23, 29 \rangle$,

where the deleted value 11 is represented by 10, the deleted values 21 and 22 are represented by 20, and the deleted value 24 is represented by 23. Because every element of the trimmed version of the list is also an element of the original version of the list, trimming can dramatically decrease the number of elements kept while keeping a close (and slightly smaller) representative value in the list for each deleted element.

The procedure TRIM trims list $L = \langle y_1, y_2, \dots, y_m \rangle$ in $\Theta(m)$ time, given L and the trimming parameter δ . It assumes that L is sorted into monotonically increasing order. The output of the procedure is a trimmed, sorted list. The procedure scans the elements of L in monotonically increasing order. A number is appended onto the returned list L' only if it is the first element of L or if it cannot be represented by the most recent number placed into L' .

TRIM (L, δ)

1 let m be the length of L

2 $L' = \langle y_1 \rangle$

3 $last = y_1$

4 **for** $i = 2$ **to** m

5 **if** $y_i > last \cdot (1 + \delta)$ // $y_i \geq last$ because L is sorted

6 append y_i onto the end of L'

```

7       $last = y_i$ 
8  return  $L'$ 

```

Given the procedure TRIM, the procedure APPROX-SUBSET-SUM on the following page implements the approximation scheme. This procedure takes as input a set $S = \{x_1, x_2, \dots, x_n\}$ of n integers (in arbitrary order), the size $n = |S|$, the target integer t , and an approximation parameter ϵ , where

$$0 < \epsilon < 1. \quad (35.23)$$

It returns a value z^* whose value is within a factor of $1 + \epsilon$ of the optimal solution.

The APPROX-SUBSET-SUM procedure works as follows. Line 1 initializes the list L_0 to be the list containing just the element 0. The **for** loop in lines 2–5 computes L_i as a sorted list containing a suitably trimmed version of the set P_i , with all elements larger than t removed. Since the procedure creates L_i from L_{i-1} , it must ensure that the repeated trimming doesn't introduce too much compounded inaccuracy. That's why instead of the trimming parameter being ϵ in the call to TRIM, it has the smaller value $\epsilon/2n$. We'll soon see that APPROX-SUBSET-SUM returns a correct approximation if one exists.

APPROX-SUBSET-SUM (S, n, t, ϵ)

```

1   $L_0 = \langle 0 \rangle$ 
2  for  $i = 1$  to  $n$ 
3       $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
4       $L_i = \text{TRIM}(L_i, \epsilon/2n)$ 
5      remove from  $L_i$  every element that is greater than  $t$ 
6  let  $z^*$  be the largest value in  $L_n$ 
7  return  $z^*$ 

```

As an example, suppose that APPROX-SUBSET-SUM is given

$$S = \langle 104, 102, 201, 101 \rangle$$

with $t = 308$ and $\epsilon = 0.40$. The trimming parameter δ is $\epsilon/2n = 0.40/80 = 0.05$. The procedure computes the following values on the indicated lines:

line 1: $L_0 = \langle 0 \rangle,$

line 3: $L_1 = \langle 0, 104 \rangle,$

line 4: $L_1 = \langle 0, 104 \rangle,$

line 5: $L_1 = \langle 0, 104 \rangle,$

line 3: $L_2 = \langle 0, 102, 104, 206 \rangle,$

line 4: $L_2 = \langle 0, 102, 206 \rangle,$

line 5: $L_2 = \langle 0, 102, 206 \rangle,$

line 3: $L_3 = \langle 0, 102, 201, 206, 303, 407 \rangle,$

line 4: $L_3 = \langle 0, 102, 201, 303, 407 \rangle,$

line 5: $L_3 = \langle 0, 102, 201, 303 \rangle,$

line 3: $L_4 = \langle 0, 101, 102, 201, 203, 302, 303, 404 \rangle,$

line 4: $L_4 = \langle 0, 101, 201, 302, 404 \rangle,$

line 5: $L_4 = \langle 0, 101, 201, 302 \rangle.$

The procedure returns $z^* = 302$ as its answer, which is well within $\epsilon = 40\%$ of the optimal answer $307 = 104 + 102 + 101$. In fact, it is within 2%.

Theorem 35.7

APPROX-SUBSET-SUM is a fully polynomial-time approximation scheme for the subset-sum problem.

Proof The operations of trimming L_i in line 4 and removing from L_i every element that is greater than t maintain the property that every element of L_i is also a member of P_i . Therefore, the value z^* returned in

line 7 is indeed the sum of some subset of S , that is, $z^* \in P_n$. Let $y^* \in P_n$ denote an optimal solution to the subset-sum problem, so that it is the greatest value in P_n that is less than or equal to t . Because line 5 ensures that $z^* \leq t$, we know that $z^* \leq y^*$. By inequality (35.1), we need to show that $y^*/z^* \leq 1 + \epsilon$. We must also show that the running time of this algorithm is polynomial in both $1/\epsilon$ and the size of the input.

As Exercise 35.5-2 asks you to show, for every element y in P_i that is at most t , there exists an element $z \in L_i$ such that

$$\frac{y}{(1 + \epsilon/2n)^i} \leq z \leq y. \quad (35.24)$$

Inequality (35.24) must hold for $y^* \in P_n$, and therefore there exists an element $z \in L_n$ such that

$$\frac{y^*}{(1 + \epsilon/2n)^n} \leq z \leq y^*,$$

and thus

$$\frac{y^*}{z} \leq \left(1 + \frac{\epsilon}{2n}\right)^n. \quad (35.25)$$

Since there exists an element $z \in L_n$ fulfilling inequality (35.25), the inequality must hold for z^* , which is the largest value in L_n , which is to say

$$\frac{y^*}{z^*} \leq \left(1 + \frac{\epsilon}{2n}\right)^n. \quad (35.26)$$

Now we show that $y^*/z^* \leq 1 + \epsilon$. We do so by showing that $(1 + \epsilon/2n)^n \leq 1 + \epsilon$. First, inequality (35.23), $0 < \epsilon < 1$, implies that

$$(\epsilon/2)^2 \leq \epsilon/2. \quad (35.27)$$

Next, from equation (3.16) on page 66, we have $\lim_{n \rightarrow \infty} (1 + \epsilon/2n)^n = e^{\epsilon/2}$. Exercise 35.5-3 asks you to show that

$$\frac{d}{dn} \left(1 + \frac{\epsilon}{2n}\right)^n > 0. \quad (35.28)$$

Therefore, the function $(1 + \epsilon/2n)^n$ increases with n as it approaches its limit of $e^{\epsilon/2}$, and we have

$$\begin{aligned} \left(1 + \frac{\epsilon}{2n}\right)^n &\leq e^{\epsilon/2} \\ &\leq 1 + \epsilon/2 + (\epsilon/2)^2 \quad (\text{by inequality (3.15) on page 66}) \\ &\leq 1 + \epsilon \quad (\text{by inequality (35.27)}) . \end{aligned} \quad (35.29)$$

Combining inequalities (35.26) and (35.29) completes the analysis of the approximation ratio.

To show that APPROX-SUBSET-SUM is a fully polynomial-time approximation scheme, we derive a bound on the length of L_i . After trimming, successive elements z and z' of L_i must have the relationship $z'/z > 1 + \epsilon/2n$. That is, they must differ by a factor of at least $1 + \epsilon/2n$. Each list, therefore, contains the value 0, possibly the value 1, and up to $\lceil \log_{1 + \epsilon/2n} t \rceil$ additional values. The number of elements in each list L_i is at most

$$\begin{aligned} \log_{1 + \epsilon/2n} t + 2 &= \frac{\ln t}{\ln(1 + \epsilon/2n)} + 2 \\ &\leq \frac{2n(1 + \epsilon/2n) \ln t}{\epsilon} + 2 \quad (\text{by inequality (3.23) on page 67}) \\ &< \frac{3n \ln t}{\epsilon} + 2 \quad (\text{by inequality (35.23), } 0 < \epsilon < 1) . \end{aligned}$$

This bound is polynomial in the size of the input—which is the number of bits $\lg t$ needed to represent t plus the number of bits needed to represent the set S , which in turn is polynomial in n —and in $1/\epsilon$. Since the running time of APPROX-SUBSET-SUM is polynomial in the lengths of the lists L_i , we conclude that APPROX-SUBSET-SUM is a fully polynomial-time approximation scheme. ■

Exercises

35.5-1

Prove equation (35.21). Then show that after executing line 4 of EXACT-SUBSET-SUM, L_i is a sorted list containing every element of P_i whose value is not more than t .

35.5-2

Using induction on i , prove inequality (35.24).

35.5-3

Prove inequality (35.28).

35.5-4

How can you modify the approximation scheme presented in this section to find a good approximation to the smallest value not less than t that is a sum of some subset of the given input list?

35.5-5

Modify the APPROX-SUBSET-SUM procedure to also return the subset of S that sums to the value z^* .

Problems

35-1 *Bin packing*

You are given a set of n objects, where the size s_i of the i th object satisfies $0 < s_i < 1$. Your goal is to pack all the objects into the minimum number of unit-size bins. Each bin can hold any subset of the objects whose total size does not exceed 1.

a. Prove that the problem of determining the minimum number of bins required is NP-hard. (*Hint:* Reduce from the subset-sum problem.)

The *first-fit* heuristic takes each object in turn and places it into the first bin that can accommodate it, as follows. It maintains an ordered list of bins. Let b denote the number of bins in the list, where b increases over the course of the algorithm, and let $\langle B_1, \dots, B_b \rangle$ be the list of bins. Initially $b = 0$ and the list is empty. The algorithm takes each object i in

turn and places it in the lowest-numbered bin that can still accommodate it. If no bin can accommodate object i , then b is incremented and a new bin B_b is opened, containing object i . Let $S = \sum_{i=1}^n s_i$.

- b.** Argue that the optimal number of bins required is at least $\lceil S \rceil$.
- c.** Argue that the first-fit heuristic leaves at most one bin at most half full.
- d.** Prove that the number of bins used by the first-fit heuristic never exceeds $\lceil 2S \rceil$.
- e.** Prove an approximation ratio of 2 for the first-fit heuristic.
- f.** Give an efficient implementation of the first-fit heuristic, and analyze its running time.

35-2 *Approximating the size of a maximum clique*

Let $G = (V, E)$ be an undirected graph. For any $k \geq 1$, define $G^{(k)}$ to be the undirected graph $(V^{(k)}, E^{(k)})$, where $V^{(k)}$ is the set of all ordered k -tuples of vertices from V and $E^{(k)}$ is defined so that $(v_1, v_2, \dots, v_k)$ is adjacent to $(w_1, w_2, \dots, w_k)$ if and only if for $i = 1, 2, \dots, k$, either vertex v_i is adjacent to w_i in G , or else $v_i = w_i$.

- a.** Prove that the size of the maximum clique in $G^{(k)}$ is equal to the k th power of the size of the maximum clique in G .
- b.** Argue that if there is an approximation algorithm that has a constant approximation ratio for finding a maximum-size clique, then there is a polynomial-time approximation scheme for the problem.

35-3 *Weighted set-covering problem*

Suppose that sets have weights in the set-covering problem, so that each set S_i in the family $\mathcal{F}$ has an associated weight w_i . The weight of a cover

$\mathcal{C}$ is $\sum_{S_i \in \mathcal{C}} w_i$. The goal is wish to determine a minimum-weight cover. (Section 35.3 handles the case in which $w_i = 1$ for all i .)

Show how to generalize the greedy set-covering heuristic in a natural manner to provide an approximate solution for any instance of the weighted set-covering problem. Letting d be the maximum size of any set S_i , show that your heuristic has an approximation ratio of $H(d) = \sum_{i=1}^d 1/i$.

35-4 *Maximum matching*

Recall that for an undirected graph G , a matching is a set of edges such that no two edges in the set are incident on the same vertex. Section 25.1 showed how to find a maximum matching in a bipartite graph, that is, a matching such that no other matching in G contains more edges. This problem examines matchings in undirected graphs that are not required to be bipartite.

- a.** Show that a maximal matching need not be a maximum matching by exhibiting an undirected graph G and a maximal matching M in G that is not a maximum matching. (*Hint:* You can find such a graph with only four vertices.)
- b.** Consider a connected, undirected graph $G = (V, E)$. Give an $O(E)$ -time greedy algorithm to find a maximal matching in G .

This problem concentrates on a polynomial-time approximation algorithm for maximum matching. Whereas the fastest known algorithm for maximum matching takes superlinear (but polynomial) time, the approximation algorithm here will run in linear time. You will show that the linear-time greedy algorithm for maximal matching in part (b) is a 2-approximation algorithm for maximum matching.

- c.** Show that the size of a maximum matching in G is a lower bound on the size of any vertex cover for G .
- d.** Consider a maximal matching M in $G = (V, E)$. Let $T = \{v \in V : \text{some edge in } M \text{ is incident on } v\}$. What can you say about the subgraph of G induced by the vertices of G that are not in T ?

- e. Conclude from part (d) that $2|M|$ is the size of a vertex cover for G .
- f. Using parts (c) and (e), prove that the greedy algorithm in part (b) is a 2-approximation algorithm for maximum matching.

35-5 *Parallel machine scheduling*

In the *parallel-machine-scheduling problem*, the input has two parts: n jobs, $J_1, J_2, \dots, J_n$, where each job J_k has an associated nonnegative processing time of p_k , and m identical machines, $M_1, M_2, \dots, M_m$. Any job can run on any machine. A *schedule* specifies, for each job J_k , the machine on which it runs and the time period during which it runs. Each job J_k must run on some machine M_i for p_k consecutive time units, and during that time period no other job may run on M_i . Let C_k denote the *completion time* of job J_k , that is, the time at which job J_k completes processing. Given a schedule, define $C_{\max} = \max \{C_j : 1 \leq j \leq n\}$ to be the *makespan* of the schedule. The goal is to find a schedule whose makespan is minimum.

For example, consider an input with two machines M_1 and M_2 , and four jobs J_1, J_2, J_3 , and J_4 with $p_1 = 2$, $p_2 = 12$, $p_3 = 4$, and $p_4 = 5$. Then one possible schedule runs, on machine M_1 , job J_1 followed by job J_2 , and on machine M_2 , job J_4 followed by job J_3 . For this schedule, $C_1 = 2$, $C_2 = 14$, $C_3 = 9$, $C_4 = 5$, and $C_{\max} = 14$. An optimal schedule runs job J_2 on machine M_1 and jobs J_1, J_3 , and J_4 on machine M_2 . For this schedule, we have $C_1 = 2$, $C_2 = 12$, $C_3 = 6$, and $C_4 = 11$, and so $C_{\max} = 12$.

Given the input to a parallel-machine-scheduling problem, let $C_{\max}^*$ denote the makespan of an optimal schedule.

- a. Show that the optimal makespan is at least as large as the greatest processing time, that is,

$$C_{\max}^* \geq \max \{p_k : 1 \leq k \leq n\} .$$

- b.** Show that the optimal makespan is at least as large as the average machine load, that is,

$$C_{\max}^* \geq \frac{1}{m} \sum_{k=1}^n p_k .$$

Consider the following greedy algorithm for parallel machine scheduling: whenever a machine is idle, schedule any job that has not yet been scheduled.

- c.** Write pseudocode to implement this greedy algorithm. What is the running time of your algorithm?
- d.** For the schedule returned by the greedy algorithm, show that

$$C_{\max} \leq \frac{1}{m} \sum_{k=1}^n p_k + \max \{p_k : 1 \leq k \leq n\} .$$

Conclude that this algorithm is a polynomial-time 2-approximation algorithm.

35-6 *Approximating a maximum spanning tree*

Let $G = (V, E)$ be an undirected graph with distinct edge weights $w(u, v)$ on each edge $(u, v) \in E$. For each vertex $v \in V$, denote by $\max(v)$ the maximum-weight edge incident on that vertex. Let $S_G = \{\max(v) : v \in V\}$ be the set of maximum-weight edges incident on each vertex, and let T_G be the maximum-weight spanning tree of G , that is, the spanning tree of maximum total weight. For any subset of edges $E' \subseteq E$, define $w(E') = \sum_{(u,v) \in E'} w(u, v)$.

- a.** Give an example of a graph with at least 4 vertices for which $S_G = T_G$.
- b.** Give an example of a graph with at least 4 vertices for which $S_G \neq T_G$.
- c.** Prove that $S_G \subseteq T_G$ for any graph G .

- d.* Prove that $w(S_G) \geq w(T_G)/2$ for any graph G .
- e.* Give an $O(V + E)$ -time algorithm to compute a 2-approximation to the maximum spanning tree.

35-7 *An approximation algorithm for the 0-1 knapsack problem*

Recall the knapsack problem from Section 15.2. The input includes n items, where the i th item is worth v_i dollars and weighs w_i pounds. The input also includes the capacity of a knapsack, which is W pounds. Here, we add the further assumptions that each weight w_i is at most W and that the items are indexed in monotonically decreasing order of their values: $v_1 \geq v_2 \geq \dots \geq v_n$.

In the 0-1 knapsack problem, the goal is to find a subset of the items whose total weight is at most W and whose total value is maximum. The fractional knapsack problem is like the 0-1 knapsack problem, except that a fraction of each item may be put into the knapsack, rather than either all or none of each item. If a fraction x_i of item i goes into the knapsack, where $0 \leq x_i \leq 1$, it contributes $x_i w_i$ to the weight of the knapsack and adds value $x_i v_i$. The goal of this problem is to develop a polynomial-time 2-approximation algorithm for the 0-1 knapsack problem.

In order to design a polynomial-time algorithm, let's consider restricted instances of the 0-1 knapsack problem. Given an instance I of the knapsack problem, form restricted instances I_j , for $j = 1, 2, \dots, n$, by removing items $1, 2, \dots, j-1$ and requiring the solution to include item j (all of item j in both the fractional and 0-1 knapsack problems). No items are removed in instance I_1 . For instance I_j , let P_j denote an optimal solution to the 0-1 problem and Q_j denote an optimal solution to the fractional problem.

- a.* Argue that an optimal solution to instance I of the 0-1 knapsack problem is one of $\{P_1, P_2, \dots, P_n\}$.

- b.** Prove that to find an optimal solution Q_j to the fractional problem for instance I_j , you can include item j and then use the greedy algorithm in which each step takes as much as possible of the unchosen item with the maximum value per pound v_i/w_i in the set $\{j+1, j+2, \dots, n\}$.
- c.** Prove that there is always an optimal solution Q_j to the fractional problem for instance I_j that includes at most one item fractionally. That is, for all items except possibly one, either all of the item or none of the item goes into the knapsack.
- d.** Given an optimal solution Q_j to the fractional problem for instance I_j , form solution R_j from Q_j by deleting any fractional items from Q_j . Let $v(S)$ denote the total value of items taken in a solution S . Prove that $v(R_j) \geq v(Q_j)/2 \geq v(P_j)/2$.
- e.** Give a polynomial-time algorithm that returns a maximum-value solution from the set $\{R_1, R_2, \dots, R_n\}$, and prove that your algorithm is a polynomial-time 2-approximation algorithm for the 0-1 knapsack problem.

Chapter notes

Although methods that do not necessarily compute exact solutions have been known for thousands of years (for example, methods to approximate the value of π), the notion of an approximation algorithm is much more recent. Hochbaum [221] credits Garey, Graham, and Ullman [175] and Johnson [236] with formalizing the concept of a polynomial-time approximation algorithm. The first such algorithm is often credited to Graham [197].

Since this early work, thousands of approximation algorithms have been designed for a wide range of problems, and there is a wealth of literature on this field. Texts by Ausiello et al. [29], Hochbaum [221], Vazirani [446], and Williamson and Shmoys [459] deal exclusively with approximation algorithms, as do surveys by Shmoys [409] and Klein and

Young [256]. Several other texts, such as Garey and Johnson [176] and Papadimitriou and Steiglitz [353], have significant coverage of approximation algorithms as well. Books edited by Lawler, Lenstra, Rinnooy Kan, and Shmoys [277] and by Gutin and Punnen [204] provide extensive treatments of approximation algorithms and heuristics for the traveling-salesperson problem.

Papadimitriou and Steiglitz attribute the algorithm APPROX-VERTEX-COVER to F. Gavril and M. Yannakakis. The vertex-cover problem has been studied extensively (Hochbaum [221] lists 16 different approximation algorithms for this problem), but all the approximation ratios are at least $2 - o(1)$.

The algorithm APPROX-TSP-TOUR appears in a paper by Rosenkrantz, Stearns, and Lewis [384]. Christofides improved on this algorithm and gave a $3/2$ -approximation algorithm for the traveling-salesperson problem with the triangle inequality. Arora [23] and Mitchell [330] have shown that if the points lie in the euclidean plane, there is a polynomial-time approximation scheme. Theorem 35.3 is due to Sahni and Gonzalez [392].

The algorithm APPROX-SUBSET-SUM and its analysis are loosely modeled after related approximation algorithms for the knapsack and subset-sum problems by Ibarra and Kim [234].

Problem 35-7 is a combinatorial version of a more general result on approximating knapsack-type integer programs by Bienstock and McClosky [55].

The randomized algorithm for MAX-3-CNF satisfiability is implicit in the work of Johnson [236]. The weighted vertex-cover algorithm is by Hochbaum [220]. Section 35.4 only touches on the power of randomization and linear programming in the design of approximation algorithms. A combination of these two ideas yields a technique called “randomized rounding,” which formulates a problem as an integer linear program, solves the linear-programming relaxation, and interprets the variables in the solution as probabilities. These probabilities then help guide the solution of the original problem. This technique was first used by Raghavan and Thompson [374], and it has had many subsequent uses. (See Motwani, Naor, and Raghavan [335] for a survey.) Several other notable ideas in the field of approximation

algorithms include the primal-dual method (see Goemans and Williamson [184] for a survey), finding sparse cuts for use in divide-and-conquer algorithms [288], and the use of semidefinite programming [183].

As mentioned in the chapter notes for Chapter 34, results in probabilistically checkable proofs have led to lower bounds on the approximability of many problems, including several in this chapter. In addition to the references there, the chapter by Arora and Lund [26] contains a good description of the relationship between probabilistically checkable proofs and the hardness of approximating various problems.

¹ When the approximation ratio is independent of n , we use the terms “approximation ratio of ρ ” and “ ρ -approximation algorithm,” indicating no dependence on n .